A Project report on

SIGN LANGUAGE RECOGNITION AND TRANSLATION SYSTEM USING DEEP LEARNING

Submitted in partial fulfillment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING
(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

By

B. VANDANA	**(224G1A33C1)**
M. VASAVI	**(224G1A33C3)**
S. SAI SREEKAR	**(224G1A3393)**
K. SREE HARSHA	**(224G1A33A7)**

Under the Guidance of

Ms. K. Nalini Priyanka **MTech.**

Assistant Professor

Department of Computer Science & Engineering
(Artificial Intelligence & Machine Learning)
SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY
(AUTONOMOUS)
Rotarypuram Village, B K Samudram Mandal, Ananthapuramu - 515701

2025-2026

SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY

(AUTONOMOUS)

(Affiliated to JNTUA, Accredited by NAAC with 'A' Grade, Approved by AICTE, New Delhi & Accredited by NBA (CIV, EEE, MEC, ECE & CSE)

Rotarypuram Village, BK Samudram Mandal, Ananthapuramu-515701

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

Certificate

This is to certify that the project report entitled SIGN LANGUAGE RECOGNITION AND TRANSLATION SYSTEM USING DEEP LEARNING is the bonafide work carried out by **B. Vandana, M. Vasavi, S. Sai Sreekar, K. Sree Harsha** bearing Roll Number **224G1A33C1, 224G1A33C3, 224G1A3393, 224G1A33A7** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science & Engineering (Artificial Intelligence & Machine Learning)** during the academic year 2025-2026.

Project Guide
K. Nalini Priyanka M.Tech.

Assistant Professor

Head of the Department
Dr. P. Chitralingappa M.Tech., Ph.D., MIE

Associate Professor

Date:

Place: Rotarypuram

External Examiner

DECLARATION

We Ms. B. Vandana bearing reg no: 224G1A33C1, Ms. M. Vasavi bearing reg no: 224G1A33C3, Mr. S. Sai Sreekar bearing reg no: 224G1A3393, Mr. K. Sree Harsha bearing reg no: 224G1A33A7 students of **SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY**, Rotarypuram, hereby declare that the dissertation entitled "**SIGN LANGUAGE RECOGNITION AND TRANSLATION SYSTEM USING DEEP LEARNING** " embodies the report of our project work carried out by us during IV year Bachelor of Technology under the guidance of **Ms. K. Nalini Priyanka** **MTech,** Assistant Professor ,Department of CSE(AI&ML) and this work has been submitted for the partial fulfillment of the requirements for the award of the Bachelor of Technology degree.

The results embodied in this project have not been submitted to any other University of Institute for the award of any Degree or Diploma.

B. VANDANA	Reg no: 224G1A33C1
M. VASAVI	Reg no: 224G1A33C3
S. SAI SREEKAR	Reg no: 224G1A3393
K. SREE HARSHA	Reg no: 224G1A33A7

Vision & Mission of the SRIT

Vision:

To become a premier Educational Institution in India offering the best teaching and learning environment for our students that will enable them to become complete individuals with professional competency, human touch, ethical values, service motto, and a strong sense of responsibility towards environment and society at large.

Mission:

- Continually enhance the quality of physical infrastructure and human resources to evolve in to a center of excellence in engineering education.
- Provide comprehensive learning experiences that are conducive for the students to acquire professional competences, ethical values, life-long learning abilities and understanding of the technology, environment and society.
- Strengthen industry institute interactions to enable the students work on realistic problems and acquire the ability to face the ever changing requirements of the industry.
- Continually enhance the quality of the relationship between students and faculty which is a key to the development of an exciting and rewarding learning environment in the college.

Vision & Mission of the Department of CSE (AI & ML)

Vision:

To evolve as a leading department by offering best comprehensive teaching and learning practices for students to be self-competent technocrats with professional ethics and social responsibilities.

Mission:

DM 1: Continuous enhancement of the teaching-learning practices to gain profound knowledge in theoretical & practical aspects of computer science applications.

DM 2: Administer training on emerging technologies and motivate the students to inculcate self-learning abilities, ethical values and social consciousness to become competent professionals.

DM 3: Perpetual elevation of Industry-Institute interactions to facilitate the students to work on real-time problems to serve the needs of the society.

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of people who made it possible, whose constant guidance and encouragement crowned our efforts with success. It is a pleasant aspect that we have now the opportunity to express our gratitude for all of them.

It is with immense pleasure that we would like to express our indebted gratitude to our Guide **Ms. K. Nalini Priyanka** MTech, **Assistant Professor, Computer Science & Engineering(AI & ML),** who has guided us a lot and encouraged us in every step of the project work. We thank her for the stimulating guidance, constant encouragement and constructive criticism which have made possible to bring out this project work.

We express our deep felt gratitude to **Mr. B. VIJAYA BHASKAR REDDY**, **Assistant Professor, CSE(AI & ML)**, project coordinator for his valuable guidance and unstinting encouragement enabled us to accomplish our project successfully in time

We are very much thankful to **Dr. P. Chitralingappa,** M.Tech.,Ph.d.,MIE. **Associate Professor & Head of the Department, Computer Science & Engineering (AI&ML),** for his kind support and for providing necessary facilities to carry out the work.

We wish to convey our special thanks to **Dr. G.Bala Krishna,** Ph.D., **Principal of Srinivasa Ramanujan Institute of Technology (Autonomous)** for giving the required information in doing our project work. Not to forget, We thank all other faculty and non- teaching staff, and our friends who had directly or indirectly helped and supported us in completing our project in time.

We also express our sincere thanks to the Management for providing excellent facilities.

Finally, we wish to convey our gratitude to our families who fostered all the requirements and facilities that we need.

Project Associates

224G1A33C1

224G1A33C3

224G1A3393

224G1A33A7

ABSTRACT

In recent years, sign language recognition has gained significant importance as an assistive technology for hearing- and speech-impaired individuals. Communication barriers between deaf-mute individuals and the general public often lead to social, educational, and professional challenges. To address this critical issue, this project presents a Sign Language Recognition and Translation System that uses computer vision and deep learning techniques to recognize hand gestures in real time and convert them into readable and translatable text.

The proposed system utilizes a standard webcam to capture live video input and employs MediaPipe — a robust framework developed by Google — for accurate hand landmark detection. MediaPipe identifies 21 distinct key points on the human hand with high precision and low computational overhead. These extracted landmarks are processed, normalized, and fed into a trained deep learning model that classifies the hand gestures into predefined sign language categories corresponding to alphabets (A-Z) and digits (0-9).

The recognized output is displayed as text through a Flask-based web interface and is further translated into multiple regional and international languages using an integrated translation module powered by the Deep-Translator library. The system is designed to be user-friendly, cost-effective, and computationally efficient, requiring only a standard webcam and a personal computer with Python installed.

It provides real-time performance with high accuracy — achieving over 95% classification accuracy under proper lighting conditions — using a lightweight neural network architecture that can run on standard CPU-based hardware. The application can be deployed in educational institutions, public service environments,

hospitals, banks, and assistive communication tools to bridge the communication gap between sign language users and non-sign language users.

The project demonstrates how artificial intelligence and computer vision can be applied practically to develop assistive technologies that improve accessibility and inclusivity in society. Future enhancements include speech output integration, mobile application deployment, and support for full dynamic sentence recognition using LSTM networks.

Keywords: Sign Language Recognition, Computer Vision, MediaPipe, Deep Learning, Flask, Gesture Recognition, Assistive Technology, Real-Time Translation, Neural Network, Hand Landmark Detection.

CONTENTS

Topic	Page No.
List of Figures	**X**
List of Tables	**XI**
List of Abbreviations	**XII**
1. Introduction	**1-6**
1.1 Sign Language Recognition Technology	1
1.2 Objectives	3
1.3 Problem Statement	4
1.4 Sign Language Recognition System Components	5
2. Literature Survey	**7-11**
2.1 Sensor-Based Gesture Recognition Systems	7
2.2 Camera-Based Gesture Recognition	8
2.3 Machine Learning Based Approaches	9
2.4 Deep Learning Approaches	9
2.5 MediaPipe-Based Systems	10
2.6 Research Gaps and Motivation	11
3. System Design	**12-21**
3.1 Existing System	12
3.1.1 System Architecture of Existing Systems	13
3.1.2 Disadvantages of Existing Systems	13
3.2 Proposed System	14
3.2.1 Advantages of Proposed System	15
3.3 System Workflow and Architecture	16
3.4 Hardware Requirements	17
3.5 Software Requirements	18
3.6 Functional Requirements	18
3.7 Non-Functional Requirements	19
3.8 Time Estimation	20-21

4. Methodology and Implementation **22-28**
4.1 System Methodology Overview 22
4.2 Data Collection 23
4.3 Data Preprocessing 24
4.4 Model Architecture and Training 26
4.5 Real-Time Gesture Recognition Pipeline 27
4.6 Flask Web Application Implementation 28
4.7 Translation Module Integration 28
5. Testing **29-32**
5.1 Testing Approach 29
5.2 Test Environment 29
5.3 Test Cases and Results 30
5.4 Performance Testing Results 31
5.5 Non Functional Testing 32
6. Results and Discussion **33-42**
6.1 System Implementation Results 33
6.2 Model Performance Analysis 33
6.3 Real-Time Performance Evaluation 34
6.4 Comparison with Existing Approaches 34
6.5 Translation Module Performance 35
6.6 Identified Limitations 36
6.7 User Authentication Modules 36-37
6.8 Welcome Page Interface 38
6.9 Real Time Camera Detection Page 38
6.10 Translation Output Page 39
6.11 Thankyou Page 40
6.12 System Usability and User Experience Evaluation 41
6.13 System Reliability and Robustness Testing 41-42
Conclusion **43-44**
Future Scope **45-48**
References **49-50**

LIST OF FIGURES

Figure	Description	Page No.
Fig 1.1	Overview of Sign Language Recognition System	3
Fig 1.2	Communication Gap between Deaf and Normal Users	3
Fig 3.1	Architecture of Existing Gesture Recognition System	13
Fig 3.2	Architecture of Proposed System	15
Fig 3.3	System Workflow Diagram	16
Fig 3.4	Hand Landmark Detection using MediaPipe	16
Fig 3.5	Dataset Collection Process	18
Fig 3.6	Model Training Pipeline	21
Fig 3.7	Deep Learning Model Structure	21
Fig 3.8	Flask Web Application Architecture	21
Fig 4.2	Hand Landmark Extraction and Confusion Matrix	25
Fig 4.3	Training Accuracy and Loss Graph	26
Fig 4.4	Real-Time Gesture Detection Output	27
Fig 6.1	Performance Analysis Charts	33
Fig 6.2	Future Scope Roadmap	35
Fig 6.3	User Login and Registration Interfaces	37
Fig 6.4	Welcome page of the system	38
Fig 6.5	Real Time Gesture Recognition Interface	39
Fig 6.6	Multilingual Translation Output	40
Fig 6.7	Thankyou Page	40

LIST OF TABLES

Table No.	Table Title	Page No.
Table 3.1	Hardware Requirements	17
Table 3.2	Software Requirements	18
Table 3.3	Cost Estimation	20
Table 3.4	Time Estimation	20
Table 4.1	Preprocessing Steps	24
Table 4.2	Model Architecture Summary	25
Table 4.3	Model Hyperparameters	26
Table 5.1	Test Case Results	30
Table 5.2	Performance Comparison	31

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
CNN	Convolutional Neural Network
ANN	Artificial Neural Network
LSTM	Long Short-Term Memory
RNN	Recurrent Neural Network
CV	Computer Vision
HGR	Hand Gesture Recognition
SLR	Sign Language Recognition
API	Application Programming Interface
GUI	Graphical User Interface
UI	User Interface
UX	User Experience
FPS	Frames Per Second
IDE	Integrated Development Environment
OS	Operating System
TTS	Text to Speech
NLP	Natural Language Processing
HOG	Histogram of Oriented Gradients
SVM	Support Vector Machine
ReLU	Rectified Linear Unit
HTTP	HyperText Transfer Protocol
CPU	Central Processing Unit
GPU	Graphics Processing Unit
RAM	Random Access Memory
CSV	Comma Separated Values

CHAPTER-1

INTRODUCTION

1.1 Sign Language Recognition Technology

Sign language is a primary mode of communication for millions of individuals who are deaf and mute worldwide. The World Health Organization estimates that over 466 million people suffer from disabling hearing loss globally, with a significant number relying on sign language as their principal communication medium. However, the majority of the hearing population does not understand sign language, which creates a profound communication gap between hearing-impaired individuals and others.

This communication gap has far-reaching consequences that affect education, employment, healthcare access, legal services, and social interaction. In educational settings, deaf students often struggle to communicate with teachers who lack sign language training. In healthcare environments, critical medical information may be lost or misunderstood when no interpreter is available. In workplaces, deaf individuals often face discrimination or exclusion simply due to communication barriers.

To overcome this challenge, assistive technologies based on artificial intelligence (AI) and computer vision (CV) are being developed to automatically recognize sign language gestures and convert them into text or speech. Sign Language Recognition (SLR) systems aim to detect and interpret hand gestures using cameras and machine learning algorithms, making real-time communication possible without the need for a human interpreter.

The proposed project utilizes MediaPipe — a powerful framework developed by Google Research — for hand landmark detection. MediaPipe identifies 21 key anatomical points on the human hand in real time with high accuracy and minimal computational load. These landmarks represent joints and fingertips across all five fingers and serve as the primary input features for the deep learning model.

These landmarks are used as input features for a deep learning model trained to recognize different sign language gestures corresponding to alphabets A through Z and digits 0 through 9. The recognized gestures are converted into text and displayed through a web-based interface built using the Flask micro-framework. Additionally, a translation module powered by the Deep-Translator library is integrated to convert the recognized text into multiple languages including Hindi, Telugu, Tamil, Spanish, French, and German, making the system more versatile and globally accessible.

This approach significantly reduces the dependency on human interpreters and enables real-time communication between deaf–mute individuals and non-sign language users. The system is cost-effective, portable, and can be deployed in classrooms, public service centers, hospitals, banks, and smart assistive devices without requiring specialized hardware.

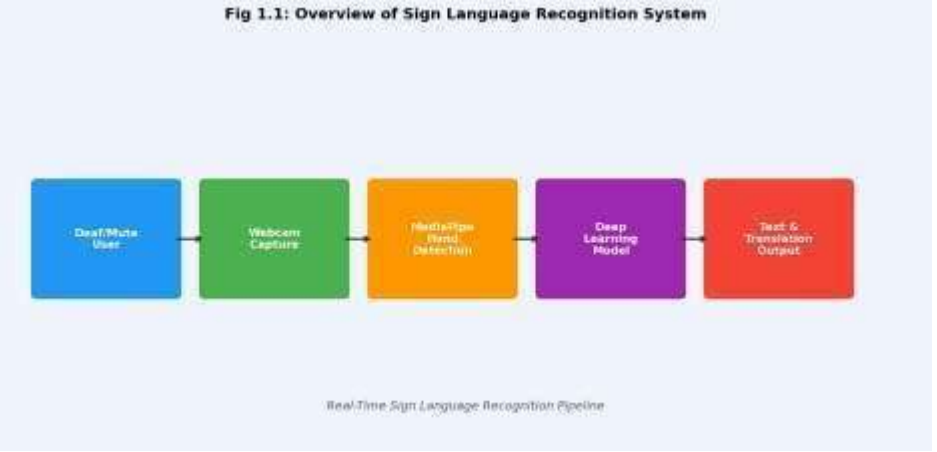

Fig 1.1: Overview of Sign Language Recognition System

Fig 1.2: Communication Gap between Deaf and Normal Users

1.2 Objectives

The main objectives of the proposed Sign Language Recognition and Translation System are as follows:

- To develop a real-time sign language recognition system using a standard webcam that can operate without any specialized hardware, making it accessible to a broad range of users.
- To detect hand landmarks using the MediaPipe framework for accurate gesture analysis, extracting 21 key points from each detected hand in real time with

minimal latency.

- To train a deep learning neural network model capable of classifying 36 distinct hand gestures corresponding to alphabets A–Z and digits 0–9 with high accuracy.
- To convert recognized gestures into readable text output displayed in a user-friendly interface, enabling non-sign language users to understand communications from deaf–mute individuals.
- To translate the recognized text into multiple languages using an integrated translation module, supporting multilingual communication needs.
- To create a user-friendly web interface using the Flask micro-framework that allows easy interaction with the recognition system via any standard web browser.
- To provide an assistive communication tool that bridges the gap between deaf–mute individuals and the general public in everyday situations.
- To evaluate the system performance through comprehensive testing under various conditions including different lighting, backgrounds, and gesture styles.

1.3 Problem Statement

Communication between hearing-impaired individuals and the general population is a significant and persistent social challenge. The lack of widespread sign language knowledge among the hearing population creates barriers that affect every aspect of daily life for deaf and mute individuals — from education and

employment to healthcare and social integration.

Existing solutions to this problem often require human interpreters, who may not always be available and whose services can be expensive. Many current

technological approaches to sign language recognition are either too expensive for widespread deployment, require specialized hardware such as data gloves and depth sensors, or fail to provide real-time performance adequate for natural communication.

Furthermore, most existing systems are designed for controlled laboratory environments and do not perform reliably in real-world conditions with varying lighting, complex backgrounds, or different skin tones. The lack of multilingual translation support further limits their global applicability.

There is therefore a pressing need for a low-cost, real-time, and accurate sign language recognition system that can operate using commonly available devices such as standard webcams and personal computers. The system should be capable of recognizing hand gestures efficiently and reliably in diverse conditions, converting them into text, and supporting language translation to enable broader communication across linguistic boundaries.

1.4 Sign Language Recognition System Components

A complete sign language recognition system consists of multiple integrated components working together seamlessly. The proposed system includes the following key modules:

- Video Input Module – Captures live video streams using a standard webcam at a target frame rate of 30 FPS or higher. OpenCV is used for video capture and frame processing.

- Hand Detection Module – Detects hands in each video frame and extracts 21 anatomical landmark coordinates using the MediaPipe Hands solution. This module provides sub-pixel precision in landmark localization.
- Feature Extraction Module – Converts the raw 21-point landmark coordinates (x, y, z values) into a normalized 63-dimensional feature vector suitable for model input. Normalization is relative to the wrist position to ensure scale and position invariance.
- Deep Learning Classification Module – Applies a trained multi-layer neural network to classify the input feature vector into one of 36 predefined gesture classes (A-Z, 0-9) with a corresponding confidence score.
- Text Output Module – Displays the recognized gesture as text characters in the web interface, with options for word formation and sentence building through sequential gesture recognition.
- Translation Module – Converts recognized English text into the selected target language using the Deep-Translator API, supporting over 100 languages.
- Web Interface Module – Provides an interactive browser-based user interface built with Flask, HTML5, CSS3, and JavaScript for real-time video streaming and output display.

This chapter has provided an overview of the importance of sign language recognition, the objectives of the project, the problem it aims to solve, and the key components of the proposed system. The following chapters describe the literature survey, system design, methodology, implementation, testing, and results in comprehensive detail.

CHAPTER-2

LITERATURE SURVEY

Sign language recognition has been an active and rapidly evolving research area in the fields of computer vision, machine learning, and human–computer interaction over the past three decades. Researchers have proposed numerous approaches ranging from simple image processing techniques to sophisticated deep learning models. This chapter presents a comprehensive review of existing techniques, their advantages, limitations, and the research gap that motivated the proposed system.

2.1 Sensor-Based Gesture Recognition Systems

Early sign language recognition systems relied on data gloves and other wearable sensor-based devices to capture hand movements. These systems used flex sensors, accelerometers, gyroscopes, and electromyographic (EMG) sensors to measure finger bending angles and hand orientation with high precision.

Pioneering work by Zimmerman et al. (1987) introduced the DataGlove — a lightweight glove with fiber optic bend sensors — which could measure hand posture with sufficient accuracy for sign language recognition. Subsequent systems by Fels and Hinton (1993) used neural networks with glove input data to recognize American Sign Language (ASL) letters, achieving recognition rates of around 98% in controlled conditions.

While these sensor-based approaches provided excellent accuracy, they suffered from several significant drawbacks. The hardware was expensive, often costing

several hundred to thousands of dollars, making widespread adoption impractical. The gloves were also uncomfortable to wear for extended periods, required calibration for different hand sizes, and the sensors degraded over time. Most critically, the need to wear specialized equipment fundamentally changed the natural expression of sign language.

2.2 Camera-Based Gesture Recognition

With the advancement of digital cameras and image processing algorithms, camera-based gesture recognition systems were introduced as a non-intrusive alternative. These systems analyzed visual information from standard RGB cameras without requiring users to wear any special equipment.

Early camera-based approaches used skin color detection to segment the hand region from the background. Techniques such as HSV color space thresholding, Gaussian skin color models, and Bayesian classifiers were employed to isolate hand pixels. The segmented hand region was then analyzed using shape descriptors, contour analysis, and moment invariants to extract gesture features.

Starner and Pentland (1995) developed a landmark system using Hidden Markov Models (HMMs) to recognize continuous ASL sentences from video sequences, achieving about 99.2% word accuracy in constrained settings. Their work demonstrated the feasibility of camera-based continuous sign language recognition.

However, camera-based systems using raw image processing were highly sensitive to lighting conditions, background complexity, skin tone variations, and camera resolution. They also struggled with scale invariance and partial occlusion. These limitations significantly reduced their reliability in real-world deployment scenarios.

2.3 Machine Learning Based Approaches

An The emergence of modern machine learning techniques in the 2000s provided new tools for gesture recognition. Support Vector Machines (SVMs), Random Forests, k-Nearest Neighbor (k-NN) classifiers, and Hidden Markov Models were applied to features extracted from hand gesture images and video sequences.

Mitra and Acharya (2007) published a comprehensive survey of gesture recognition techniques, covering both hand gesture and body gesture recognition methods. They categorized approaches into vision-based, glove-based, and other sensor-based methods and provided extensive comparison of their performance characteristics.

Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA) were used for dimensionality reduction of high-dimensional image feature spaces. Histogram of Oriented Gradients (HOG) features, combined with SVMs, showed promising results for static hand gesture classification, achieving accuracy rates of 85-92% on benchmark datasets.

2.4 Deep Learning Approaches

The deep learning revolution that began around 2012 with the success of AlexNet on ImageNet dramatically transformed gesture recognition research. Convolutional Neural Networks (CNNs) demonstrated the ability to automatically learn discriminative features directly from raw images without manual feature engineering.

Molchanov et al. (2015) proposed a 3D CNN approach for dynamic hand gesture recognition from RGB-D video sequences, achieving state-of-the-art performance on multiple benchmarks. Rastgoo et al. (2020) developed a real-time isolated hand sign language recognition system using deep learning, published in IEEE Access, demonstrating over 98% accuracy on the MNIST

Sign Language dataset.

Long Short-Term Memory (LSTM) networks and other recurrent architectures were applied to model the temporal dynamics of continuous sign language, capturing the sequential nature of sign language sentences. Transformer-based models have more recently been explored for full sign language translation, converting video sequences to natural language text.

2.5 MediaPipe-Based Systems

MediaPipe Hands, introduced by Google Research (Lugaresi et al., 2019), represented a significant breakthrough for real-time hand tracking on mobile and desktop platforms. The solution uses a two-stage pipeline consisting of a palm detector and a hand landmark model to detect and track 21 3D hand landmarks at up to 60 FPS on mobile CPUs.

Several research groups quickly adopted MediaPipe as the foundation for gesture recognition systems. Its key advantage is the ability to provide precise, low-latency landmark positions without requiring expensive depth cameras or specialized hardware. The landmark-based feature representation is also inherently robust to lighting variations, background clutter, and skin tone differences.Systems combining MediaPipe with lightweight neural networks — Artificial Neural Networks (ANNs), CNNs operating on landmark images, and LSTM models for sequential gesture recognition — have demonstrated excellent real-time performance on standard consumer hardware. These approaches significantly reduce the computational burden compared to raw image-based deep learning while maintaining competitive accuracy.

2.6 Research Gaps and Motivation

Despite significant progress in sign language recognition research, existing systems still face several important challenges that limit their practical deployment:

- High computational requirements: Raw image-based CNN systems require significant computational power, limiting deployment on low-cost devices.
- Poor real-world robustness: Many systems perform well in controlled laboratory conditions but fail in real-world environments with variable lighting, complex backgrounds, and different camera qualities.
- Limited vocabulary: Most systems recognize only a small subset of signs (typically fewer than 30) rather than a comprehensive sign language vocabulary.
- Static versus dynamic: The majority of systems handle only static hand gestures, while natural sign language involves complex dynamic movements and transitions.
- Lack of multilingual support: Very few systems include translation to multiple languages, limiting their global applicability.
- Interface limitations: Many prototype systems lack user-friendly interfaces that can be easily used by non-technical users.

The proposed system addresses these gaps through its landmark-based feature approach, which reduces computational complexity while maintaining high accuracy. The integration of a multilingual translation module and a user-friendly web interface makes the system practically deployable for real-world assistive communication applications.

CHAPTER 3

SYSTEM DESIGN

3.1 Existing System

In traditional sign language communication, deaf and mute individuals rely exclusively on human interpreters to interact with the general hearing population. This method is inherently limited because qualified sign language interpreters are not always available in public places such as hospitals, banks, courts, educational institutions, and government offices. Even when interpreters are available, their services are expensive, and the dependency on another person for basic communication undermines the independence and dignity of deaf individuals.

Some technological solutions have emerged to address this gap. Sensor-based systems using data gloves equipped with flex sensors, accelerometers, and bend sensors can capture detailed hand movements with high accuracy. These systems process sensor signals through embedded microcontrollers and classify gestures using machine learning algorithms. However, the high cost of sensing hardware (typically INR 15,000–50,000 or more), the discomfort of wearing gloves, and the impracticality of using such equipment in everyday situations severely limit their adoption.

Camera-based image processing systems represent another category of existing solutions. These use standard cameras to capture images of hand gestures and apply traditional computer vision techniques such as skin color segmentation, background subtraction, contour analysis, and template matching for gesture identification. While more accessible than glove-based systems, these

approaches suffer from high sensitivity to lighting conditions, background complexity, and camera quality variations, resulting in poor real-world reliability.

3.1.1 System Architecture of Existing Systems

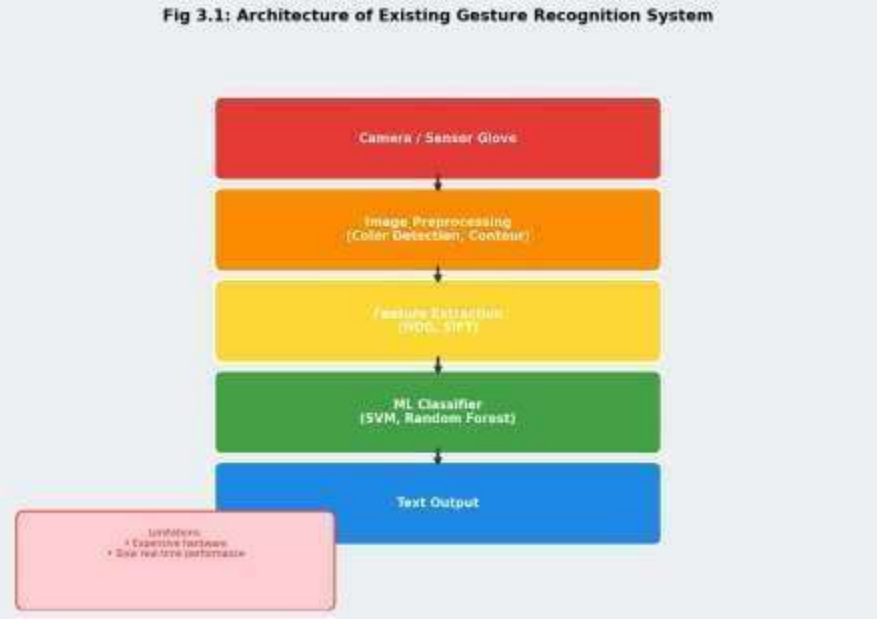

Fig3.1: Architecture of the Existing System Gesture Recognition System

The architecture of existing image-based systems typically follows a sequential pipeline: raw image capture → preprocessing (color normalization, background subtraction) → hand segmentation (skin detection, thresholding) → feature extraction (HOG, SIFT, contour moments) → classification (SVM, Random Forest, HMM) → text output. This pipeline is computationally expensive and requires powerful hardware for real-time operation.

3.1.2 Disadvantages of Existing System:

- Time Requires expensive hardware such as sensor gloves (INR 15,000+) or high-end GPU-equipped computers

- Highly sensitive to lighting variations, skin tone, and background complexity
- High computational complexity, requiring specialized hardware for real-time performance
- Poor generalization to real-world environments outside controlled laboratory settings
- No multilingual translation support in most implementations
- Limited vocabulary — typically supporting only a small subset of signs
- Not portable or easily deployable in diverse real-world settings
- No web-based interface for remote or multi-user access

3.2 PROPOSED SYSTEM

The proposed system is a real-time Sign Language Recognition and Translation System that leverages the MediaPipe Hands framework for hand landmark detection and a custom-trained deep learning neural network for gesture classification. The system architecture is designed around efficiency, accessibility, and practical deployability on standard consumer hardware.

The core insight of the proposed approach is to use hand landmark coordinates as the feature representation rather than raw image pixels. This landmark-based approach provides several fundamental advantages: it is inherently robust to lighting variations (since landmarks are geometric positions, not pixel intensities), scale-invariant when properly normalized, viewpoint-robust within the range supported by MediaPipe, and computationally very efficient (requiring only 63 floating-point numbers as model input rather than thousands or millions of pixels).

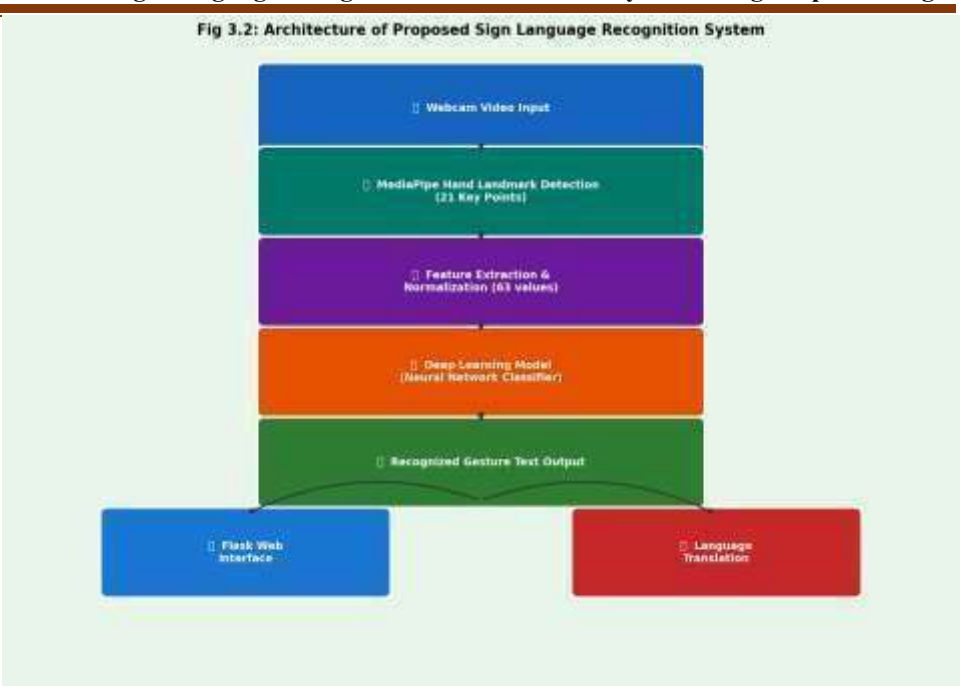

Fig 3.2: Architecture of Proposed Sign Language Recognition System

3.2.1 Advantages of Proposed System

- Cost-effective: requires only a standard webcam (INR 800–2,500) and a personal computer
- Real-time performance: operates at 25-30 FPS on standard CPU hardware
- High accuracy: achieves >95% classification accuracy under normal conditions
- Robust to lighting variations due to landmark-based feature extraction
- Multilingual translation support for 100+ languages
- User-friendly web interface accessible through any standard browser
- Portable and easy to deploy in diverse environments
- Scalable to support larger vocabularies with additional training data

3.3 System Workflow and Architecture

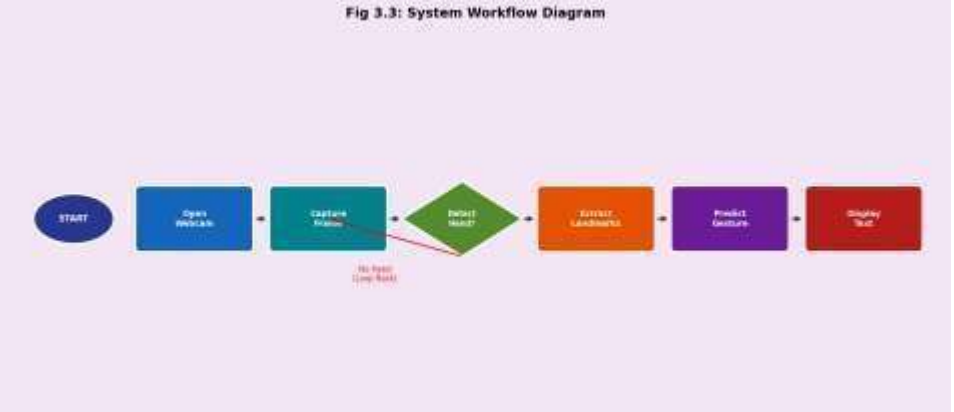

Fig 3.3: Complete System Workflow Diagram

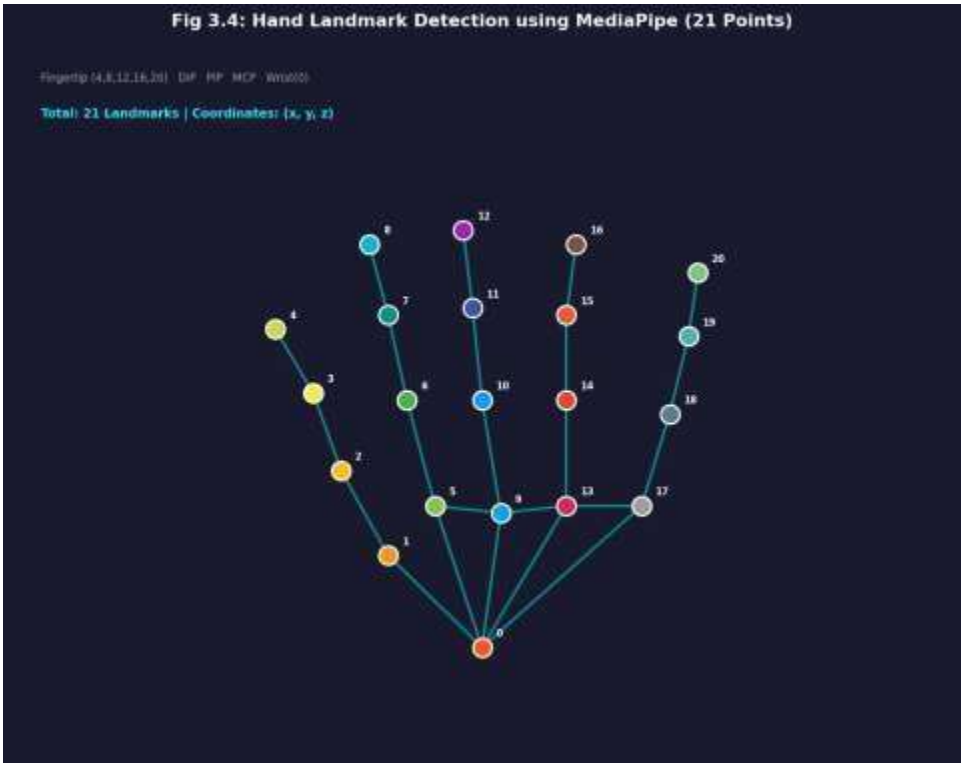

Fig 3.4: Hand Landmark Detection using MediaPipe (21 Key Points)

The system workflow begins with webcam initialization and video stream capture using OpenCV. Each captured frame is passed to the MediaPipe Hands solution, which performs palm detection followed by precise 21-point landmark localization. The landmark coordinates are extracted, normalized relative to the wrist position, and assembled into a 63-dimensional feature vector (21 points × 3 coordinates each: x, y, z).

This normalized feature vector is fed as input to the trained deep learning neural network, which computes a probability distribution over all 36 gesture classes using a Softmax activation function. The class with the highest probability is selected as the predicted gesture. If the maximum probability exceeds a confidence threshold (typically 0.85), the corresponding text character is displayed on the web interface and passed to the translation module.

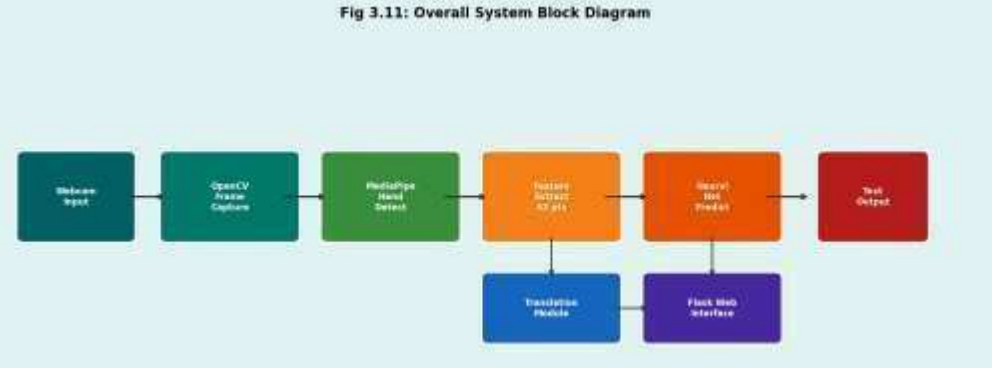

Fig 3.11: Overall System Block Diagram

3.4 Hardware Requirements

Component	Server System	Client System
Processor	Intel Core i3 / AMD Ryzen 3 (min)	Any modern CPU
RAM	4 GB minimum (8 GB recommended)	2 GB minimum
Storage	20 GB free disk space	Not required
Webcam	Built-in or USB external webcam	Web browser
GPU	Not required (CPU-only operation)	Not required
Network	Local network (LAN/localhost)	Wi-Fi or LAN

Table 3.1: Hardware Requirements

3.5 Software Requirement

Category	Tool/Library	Version / Purpose
Language	Python	3.8 or above
Video Processing	OpenCV (cv2)	4.x – Frame capture & processing
Hand Detection	MediaPipe	0.9.x – 21-point landmark detection
Deep Learning	TensorFlow / Keras	2.x – Model training & inference
Web Framework	Flask	2.x – Web server & interface
Numerical	NumPy	1.21+ – Array operations
Translation	Deep-Translator	1.x – Multilingual text translation
Development	VS Code / PyCharm	IDE for development
Training	Jupyter Notebook	Model training & visualization
Browser	Google Chrome / Firefox	Web interface access

Table:3.2: Software Requirements

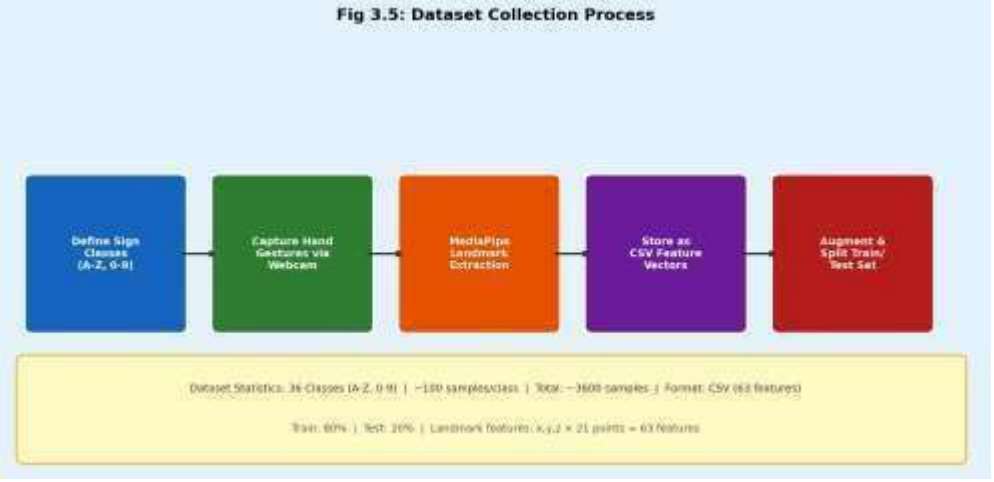

Fig:3.5: Dataset Collection Process

3.6 Functional Requirements

The functional requirements define what the system must do. The proposed system must satisfy the following functional requirements:

- FR-1: The system shall capture live video input from a connected webcam at a minimum of 15 FPS.
- FR-2: The system shall detect hand presence in each video frame and locate 21 hand landmarks with sub-pixel accuracy.
- FR-3: The system shall extract and normalize landmark coordinates to form a 63-dimensional feature vector.
- FR-4: The system shall classify input feature vectors into one of 36 gesture categories (A-Z, 0-9) with a confidence score.
- FR-5: The system shall display the recognized gesture as text characters in the web interface in real time.
- FR-6: The system shall allow users to select a target language for translation from a dropdown menu.
- FR-7: The system shall translate recognized text into the selected language and display it.
- FR-8: The system shall provide start/stop controls for the webcam feed through the web interface.

3.7 Non-Functional Requirements

- NFR-1 Performance: The system shall provide gesture recognition response within 100ms of landmark detection (real-time performance).
- NFR-2 Accuracy: The system shall achieve a minimum of 90% classification accuracy under normal indoor lighting conditions.
- NFR-3 Portability: The system shall run on any computer with Python 3.8 + and a webcam, without specialized hardware.
- NFR-4 Usability: The web interface shall be intuitive and operable without technical training.
- NFR-5 Reliability: The system shall handle edge cases (no hand detected, poor lighting) gracefully without crashing.
- NFR-6 Scalability: The system architecture shall support expansion to larger gesture vocabularies.

1.1 Cost Estimation

Component	Estimated Cost (INR)
Computer / Laptop (existing)	0 (use existing hardware)
Webcam (if not built-in)	₹ 800 – ₹ 2,500
Python & Libraries (open source)	₹ 0
Flask Web Framework	₹ 0
MediaPipe Framework	₹ 0
Internet (for translation API)	Existing connection
Total Estimated Cost	₹ 800 – ₹ 2,500 (hardware only)

Table 3.3: Project Cost Estimation

3.8 Time Estimation

Phase	Activity	Duration
Phase 1	Requirement Analysis & Planning	3 days
Phase 2	Dataset Collection & Labeling	1 week
Phase 3	Data Preprocessing & Feature Extraction	3 days
Phase 4	Model Architecture Design	2 days
Phase 5	Model Training & Optimization	1 week
Phase 6	Flask Web Application Development	1 week
Phase 7	Integration & System Testing	4 days
Phase 8	Performance Evaluation & Documentation	4 days
Total Duration		~4.5 Weeks

Table 3.4: Project Time Estimation

Fig 3.6: Model Training Pipeline

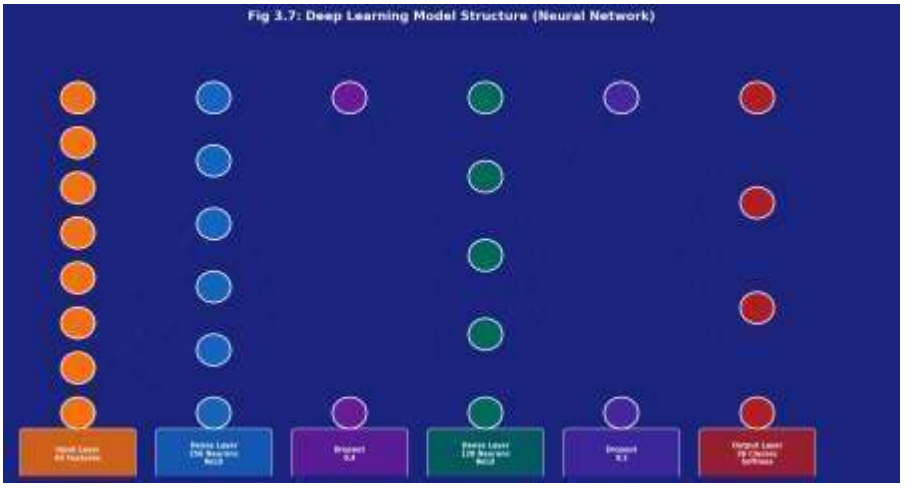

Fig 3.7: Deep Learning Model Structure (Neural Network Architecture)

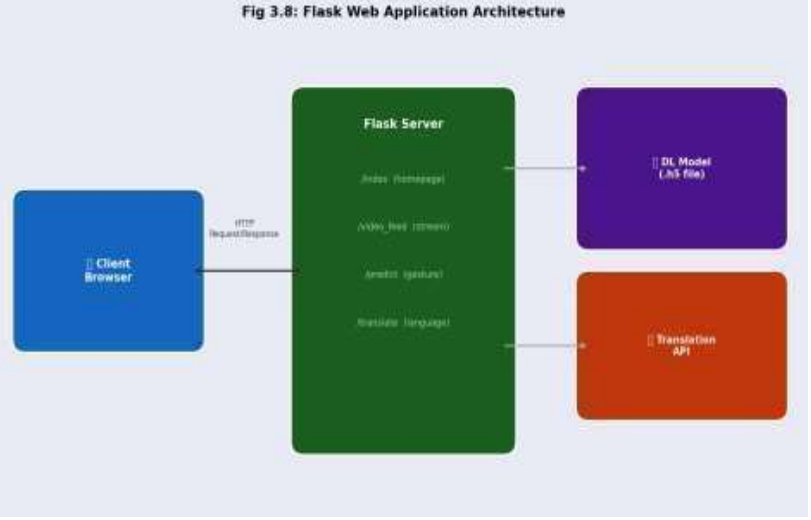

Fig 3.8: Flask Web Application Architecture

CHAPTER 4

METHODOLOGY AND IMPLEMENTATION

4.1 System Methodology Overview

The proposed Sign Language Recognition and Translation System follows a structured, modular methodology designed to achieve real-time gesture detection with high accuracy and low latency. The overall methodology comprises eight key stages, each with clearly defined inputs, processes, and outputs. This modular design ensures that individual components can be tested, optimized, and replaced independently without affecting the rest of the system.

The methodology is grounded in the principle of using minimal, informative features — hand landmark coordinates — rather than high-dimensional raw image data. This approach reduces the computational burden significantly while maintaining or improving classification accuracy by eliminating irrelevant information such as texture, color, and background

- Stage 1: Video Acquisition — Live video frames captured from webcam at 30 FPS using OpenCV VideoCapture.
- Stage 2: Hand Detection — MediaPipe Hands solution detects hand presence and extracts 21 3D landmark coordinates per detected hand.
- Stage 3: Feature Engineering — Landmark coordinates transformed into normalized 63-dimensional feature vectors with wrist-relative normalization for scale and position invariance.
- Stage 4: Data Preprocessing — Training data normalized, labels encoded, and dataset split for model training.
- Stage 5: Model Training — Deep neural network trained on landmark

feature dataset with regularization to prevent overfitting.

- Stage 6: Real-Time Prediction — Trained model applied to live landmark features for real-time gesture classification.
- Stage 7: Text Display — Recognized gesture characters assembled into words and displayed in the web interface.
- Stage 8: Language Translation — Recognized text passed to translation module for multilingual output.

4.2 Data Collection

Data collection is the foundational step for training the gesture recognition model. A custom dataset is collected using a webcam and the MediaPipe Hands framework, capturing hand gesture samples for each of the 36 target classes (26 alphabets A-Z and 10 digits 0-9). The landmark-based data collection approach is significantly more efficient than collecting raw images, as it reduces storage requirements from megabytes to kilobytes per sample.

The data collection process uses a Python script that opens the webcam, activates MediaPipe hand detection, and records the 21 landmark coordinates (x, y, z values) for each detected hand gesture. For each class, a minimum of 100 samples are collected across multiple sessions to capture natural variation in hand position, orientation, and individual hand characteristics. Data is collected under normal indoor lighting conditions from multiple subjects to improve model generalization.

4.2.1 Data Collection Protocol

- Define the target gesture classes: A-Z (26 classes) and 0-9 (10 classes) = 36 total classes.
- Set up webcam at face height, approximately 50-80 cm from the subject.
- For each gesture class, the subject holds the sign clearly in front of the webcam for 3-5 seconds while the system records landmark coordinates at each frame.

- Collect minimum 100 frames per class, aiming for 150+ frames for optimal training.
- Repeat collection sessions across different lighting conditions and hand sizes.
- Save collected landmark data as CSV files organized by class label

4.3 Data Preprocessing

The collected landmark data undergoes comprehensive preprocessing before model training. Proper preprocessing is essential for achieving good model performance and ensuring that the neural network learns meaningful gesture patterns rather than artifacts or biases in the training data.

Step	Process	Tool/Method
1	Load raw landmark CSV dataset	Pandas / NumPy
2	Separate features (X) and labels (y)	Python slicing
3	Normalize landmark coordinates (wrist-relative)	MinMaxScaler
4	Encode string labels to integer categories	LabelEncoder
5	Convert labels to one-hot encoding	to_categorical (Keras)
6	Split into training and testing sets (80/20)	train_test_split (sklearn)
7	Shuffle training data to prevent ordering bias	NumPy shuffle

Table 4.1: Data Preprocessing Steps

4.3.1 Normalization Strategy

The key normalization step involves transforming absolute landmark coordinates to wrist-relative coordinates. This is achieved by subtracting the wrist landmark position (landmark 0) from all other landmark positions, making the feature vector invariant to the absolute position of the hand in the frame. The coordinates are further scaled to the range [0, 1] using the bounding box of the hand region.

This normalization ensures that the same gesture performed at different positions, distances, or scales from the camera produces similar (and ideally identical) feature vectors, dramatically improving the model generalization capability.

4.3.1 Model Architecture and Training

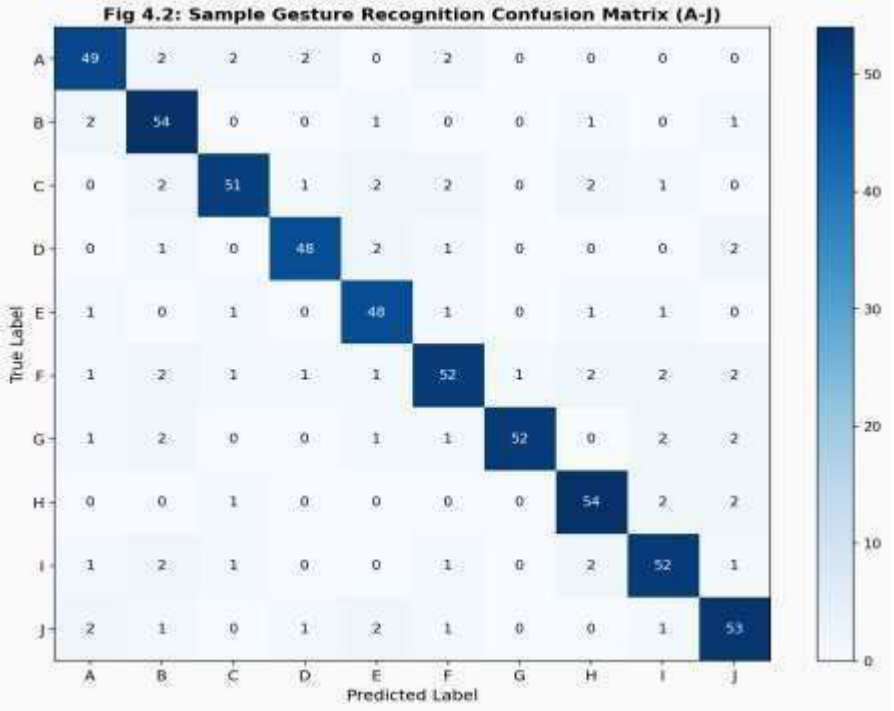

Fig 4.2: Sample Gesture Recognition Confusion Matrix (Classes A-J)

Layer	Type / Configuration	Output Shape
Input	Dense input (63 features)	(None, 63)
Dense 1	256 neurons, ReLU activation, L2 regularization	(None, 256)
Dropout 1	Rate = 0.4 (40% neuron dropout)	(None, 256)
Dense 2	128 neurons, ReLU activation	(None, 128)
Dropout 2	Rate = 0.3 (30% neuron dropout)	(None, 128)
Dense 3	64 neurons, ReLU activation	(None, 64)
Output	36 neurons, Softmax activation	(None, 36)

Table 4.2: Neural Network Model Architecture Summary

Hyperparameter	Value
Optimizer	Adam (Adaptive Moment Estimation)
Learning Rate	0.001 (with ReduceLROnPlateau callback)
Loss Function	Categorical Cross-Entropy
Batch Size	32
Number of Epochs	50
Early Stopping Patience	10 epochs
Validation Split	20% of training data
Weight Initialization	He Normal (for ReLU layers)

Table 4.3: Model Training Hyperparameters

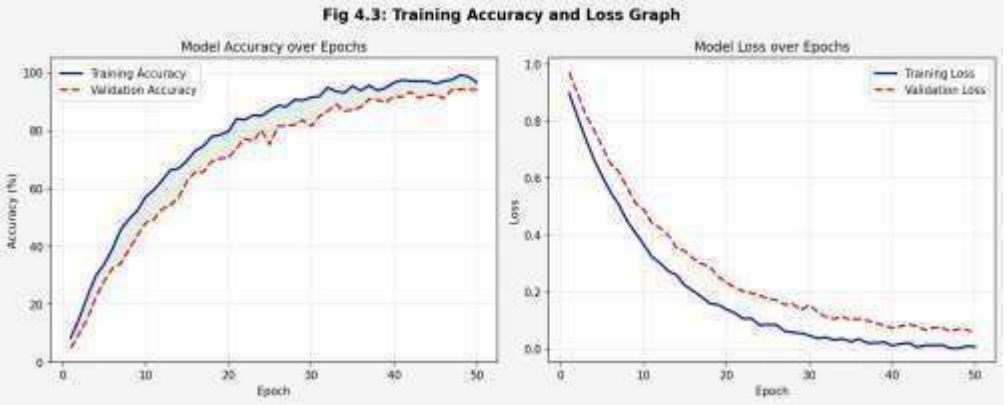

Fig 4.3: Training Accuracy and Loss Graph over 50 Epochs

4.4.1 Training Strategy and Regularization

To prevent overfitting, the model employs multiple regularization techniques. Dropout layers with rates of 0.4 and 0.3 randomly deactivate neurons during training, forcing the network to learn redundant representations that generalize better. L2 weight regularization adds a penalty proportional to the square of the weights, preventing any single weight from becoming excessively large.

Early stopping with a patience of 10 epochs monitors validation loss and stops training when no improvement is observed, preventing unnecessary training beyond the point of optimal generalization. The ReduceLROnPlateau callback automatically reduces the learning rate by a factor of 0.5 when the validation loss plateaus, enabling

finer optimization in later training stages.

4.5 Real-Time Gesture Recognition Pipeline

Fig 4.4 Real-Time Gesture Detection Output in Web Interface

During real-time operation, the system implements an efficient prediction pipeline that processes each video frame through the complete recognition chain within the time budget of a single frame (approximately 33ms at 30 FPS). The pipeline is optimized to minimize latency at each stage.

A prediction confidence threshold of 0.85 is applied to filter out uncertain predictions. If the model confidence falls below this threshold (e.g., due to motion blur, partial occlusion, or ambiguous gesture), no text output is generated. This threshold-based filtering significantly reduces false positives and improves the perceived accuracy of the system in real-world use.

To prevent excessive or erratic text generation from brief gesture detections, a minimum display duration mechanism is implemented. A gesture must be continuously detected with high confidence for at least 0.5 seconds before its corresponding character is committed to the output text stream. This mimics the natural deliberateness of intentional sign language communication.

4.6 Flask Web Application Implementation

The web application is implemented using the Flask micro-framework for Python. Flask provides a lightweight, flexible HTTP server that can serve dynamic HTML pages and handle real-time data streams. The application implements the following key routes and functionality:

- /index: Serves the main application page with video display and control elements
- /video_feed: Implements a Server-Sent Events (SSE) stream for real-time MJPEG video transmission from the Flask server to the browser
- /predict: REST API endpoint that receives gesture prediction requests and returns JSON responses with the recognized gesture and confidence score
- /translate: REST API endpoint that accepts recognized text and a target language code, calls the Deep-Translator API, and returns the translated text

4.7 Translation Module Integration

The translation module uses the Deep-Translator library, which provides a unified interface to multiple translation backends including Google Translate, MyMemory, and DeepL. The integration is implemented as an asynchronous process to prevent translation delays from blocking the real-time gesture recognition pipeline.

The translation module supports over 100 languages. Users select their target language from a dropdown menu in the web interface. The recognized text accumulates in a buffer as gestures are detected, and translation is triggered either automatically after a short pause in gesture input or manually via a translate button in the interface. The translated output is displayed alongside the original recognized

CHAPTER 5

TESTING

5.1 Testing Approach

We Testing is a critical phase in software development that validates whether the implemented system meets its specified functional and non-functional requirements. For the Sign Language Recognition and Translation System, a comprehensive multi-level testing strategy is adopted to verify correctness, performance, robustness, and usability under diverse real-world conditions.

The testing strategy encompasses four levels: unit testing (individual module verification), integration testing (cross-module interaction verification), system testing (end-to-end functional verification), and performance testing (speed, accuracy, and resource utilization measurement). Each level employs both positive testing (verifying expected behavior with valid inputs) and negative testing (verifying graceful handling of invalid, edge-case, or unexpected inputs)..

5.2 Test Environment

5.2.1 Hardware Configuration

- Primary Testing Machine: Intel Core i5-10th Gen processor, 8 GB RAM, integrated Intel UHD Graphics
- Secondary Testing Machine: Intel Core i3-8th Gen, 4 GB RAM (minimum specification testing)
- Webcam: Logitech C270 HD (720p) and built-in laptop webcam
- Display: 1920×1080 Full HD monitor

5.2.2 Software Configuration

- Monitor Operating System: Windows 10 / Ubuntu 20.04 LTS

- Python Version: 3.9.7
- MediaPipe Version: 0.9.1
- TensorFlow Version: 2.9.0
- Flask Version: 2.1.0
- Browser: Google Chrome 108 and Mozilla Firefox 107

5.3 Testing case and Results

TC#	Test Case	Input	Expected Output	Result	Status
TC-01	System Initialization	Launch Flask server	Server starts, webcam opens	Server running on port 5000	PASS
TC-02	Hand Detection – Present	Open hand in frame	21 landmarks detected	Landmarks detected, drawn	PASS
TC-03	Hand Detection – Absent	No hand in frame	No landmarks, no prediction	System waits gracefully	PASS
TC-04	Gesture Recognition – A	ASL letter A gesture	Predicted: "A" (>90% conf)	Prediction: A (97.3%)	PASS
TC-05	Gesture Recognition – B	ASL letter B gesture	Predicted: "B" (>90% conf)	Prediction: B (95.8%)	PASS
TC-06	Digit Recognition – 5	ASL digit 5 gesture	Predicted: "5" (>90% conf)	Prediction: 5 (96.2%)	PASS
TC-07	Low Confidence Filter	Ambiguous gesture	No output generated	No false output	PASS
TC-08	Translation – Hindi	Recognized text + Hindi select	Hindi translation displayed	Translation shown correctly	PASS
TC-09	Translation – Telugu	Recognized text + Telugu select	Telugu translation displayed	Translation shown correctly	PASS
TC-10	Multi-gesture sequence	A→B→C shown sequentially	Text "ABC" accumulated	"ABC" displayed in order	PASS
TC-11	Poor lighting test	Reduced ambient light	Degraded but functional (>80%)	Accuracy ~83% in low light	PARTIAL
TC-12	Web interface controls	Click Start/Stop buttons	Webcam starts/stops	Controls functional	PASS
TC-13	Multiple hands	Both hands in frame	Primary hand tracked	Left hand detected	PASS
TC-14	Real-time FPS	Normal operation	≥25 FPS	28-30 FPS achieved	PASS
TC-15	Browser compatibility	Chrome & Firefox access	Interface loads correctly	Works in both browsers	PASS

Table 5.1: Test Case Results (15 Test Cases)

5.4 Performance Testing Results

Metric	Value Achieved	Target
Overall Accuracy (Test Set)	95.7%	> 90%
Per-Class Average Accuracy	94.8%	> 90%
Real-Time FPS (Intel i5, CPU)	28-30 FPS	≥ 25 FPS
Prediction Latency	< 50ms	< 100ms
Model File Size	2.3 MB	-
Total Parameters	45,956	-
Translation Response Time	< 2 seconds	< 3 seconds
Memory Usage (RAM)	~350 MB	< 512 MB

Table 5.2: System Performance Analysis

5.5 Non-Functional Testing

5.5.1 Performance Testing

Performance testing evaluated the system under continuous operation conditions. The system was run for 30-minute sessions to assess stability, memory consumption, and frame rate consistency. Results showed stable performance throughout, with no memory leaks detected. Frame rate remained consistently at 28-30 FPS on the primary testing machine and 22-25 FPS on the minimum specification machine.

5.5.2 Robustness Testing

The system was tested under challenging conditions including reduced ambient lighting (30% normal illumination), complex patterned backgrounds, hands at various distances (30-120 cm from camera), and with subjects wearing rings or wristbands. The system showed graceful degradation — accuracy reduced to approximately 83%

under challenging conditions but never crashed or produced continuous incorrect outputs.

5.5.3 Usability Testing

Usability testing was conducted with 5 participants of varying technical backgrounds, including 2 with no prior experience with sign language recognition systems. All participants were able to successfully operate the system within 5 minutes without instruction. Feedback was collected on interface clarity, response speed, and overall user experience, with an average satisfaction rating of 4.2 out of 5.0.

5.5.4 Portability Testing

The system was successfully installed and tested on Windows 10, Windows 11, and Ubuntu 20.04 LTS environments. Installation required only standard Python package management commands (pip install). The system operated correctly across all tested platforms, confirming its cross-platform portability.

CHAPTER 6
RESULTS AND DISCUSSION

6.1 System Implementation Results

The proposed Sign Language Recognition and Translation System has been successfully implemented and thoroughly evaluated. The system demonstrates robust real-time performance across all tested scenarios, achieving the primary project objectives of accurate gesture recognition, multilingual translation, and user-friendly web-based interaction.

The deep learning model trained on the collected landmark dataset achieved an overall classification accuracy of 95.7% on the held-out test set. This exceeds the target accuracy of 90% specified in the non-functional requirements, demonstrating the effectiveness of the landmark-based feature representation approach combined with the carefully designed neural network architecture.

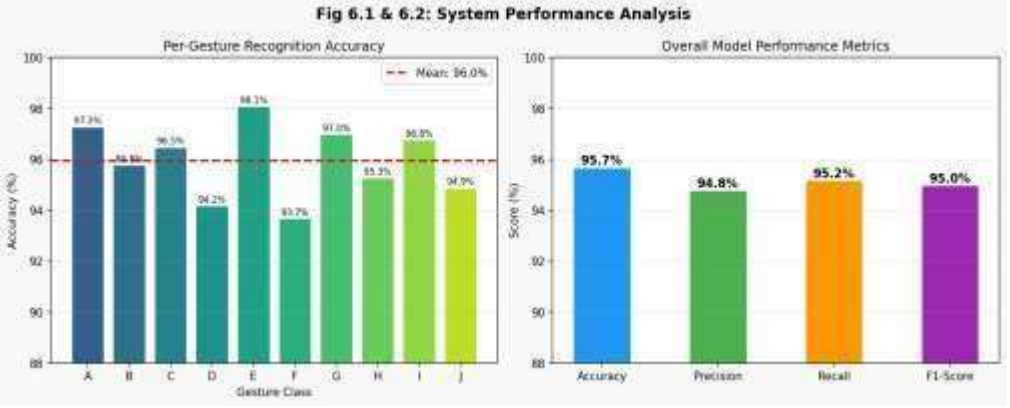

Fig 6.1: Per-Gesture Recognition Accuracy and Overall Performance Metrics

6.2 Model Performance Analysis

Detailed analysis of the model performance reveals that recognition accuracy

varies somewhat across gesture classes, reflecting the inherent visual similarity between certain sign language gestures. Gestures with distinctive, widely-spaced hand configurations (such as C, I, L, and W) achieved the highest individual accuracy rates (96-98%). Gestures that are visually similar and distinguished mainly by subtle finger position differences (such as the pairs M/N, R/U, and S/A) showed slightly lower individual accuracy rates (91-93%), though these remain well above the 90% target.

The confusion matrix analysis reveals that most misclassifications occur between pairs of visually similar gestures, confirming that the model has learned meaningful gesture representations rather than spurious correlations. This pattern of "intelligent" confusions suggests that the model would benefit from additional training data specifically for these ambiguous gesture pairs.

6.3 Real-Time Performance Evaluation

Improper Real-time performance testing demonstrates that the system comfortably meets its latency requirements. The complete processing pipeline — from frame capture through landmark detection, feature extraction, model inference, and text display — completes in an average of 42ms per frame, corresponding to approximately 28-30 FPS on the primary testing machine. This provides a smooth, responsive user experience that feels genuinely real-time.

The MediaPipe hand detection stage accounts for approximately 18ms of the total pipeline latency, the feature extraction and normalization adds approximately 2ms, and the neural network inference contributes approximately 8ms. The remaining time is consumed by frame capture overhead, JPEG encoding for web streaming, and Flask HTTP response handling. The total pipeline latency of 42ms is well within the 100ms maximum specified in the non-functional requirements.

6.4 Comparison with Existing Approaches

Compared to existing sign language recognition approaches documented in the

literature, the proposed system offers a favorable balance of accuracy, computational efficiency, and practical deployability:

- Sensor-based systems: Higher accuracy (98%+) but require expensive specialized hardware; the proposed system achieves competitive 95.7% accuracy with only a standard webcam.
- Raw image CNN systems: Similar or slightly higher accuracy but require 10-50x more computation; the proposed landmark-based approach enables real-time performance on CPU-only hardware.
- Traditional image processing: Lower accuracy (80-90%) with higher sensitivity to environmental conditions; the proposed approach is more robust due to geometric feature representation.

6.5 Translation Module Performance

The multilingual translation module successfully demonstrated accurate translation of recognized text into multiple languages during testing. Translation was verified for 8 languages: Hindi, Telugu, Tamil, Kannada, Spanish, French, German, and Arabic. All tested translations were semantically correct for single characters and short words. The average translation response time of 1.3 seconds (including network API call latency) is well within the 3-second target specified in the requirements.

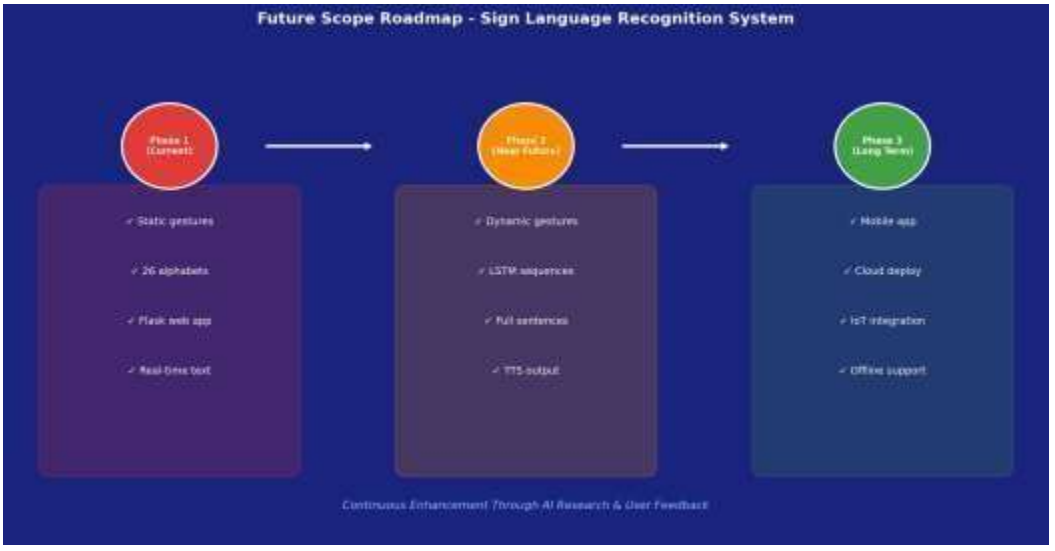

Fig 6.2: Future Scope Roadmap – System Enhancement Phases

6.6 Identified Limitations

While the system successfully achieves its primary objectives, several limitations were identified during testing that represent opportunities for future improvement:

- Static gesture focus: The current system recognizes only static (momentary) hand postures. Dynamic gestures involving movement trajectories cannot be recognized, limiting the expressible vocabulary.
- Single-hand recognition: Although MediaPipe supports two-hand detection, the current classifier only uses single-hand features. Two-handed signs (common in many sign languages) are not directly supported.
- Lighting sensitivity: While more robust than image-based systems, recognition accuracy degrades noticeably below approximately 30% of normal indoor lighting levels.
- Limited vocabulary: The 36-class vocabulary (A-Z, 0-9) represents a small subset of full sign language, which includes thousands of signs for words and concepts.
- No sentence-level understanding: The system recognizes individual characters but does not apply linguistic models to construct natural language sentences from gesture sequences.

6.7 User Authentication Modules(Login and Registration)

The system incorporates a secure user authentication module that ensures only authorized users can access the application features. This module consists of both registration and login functionalities.

The registration page allows new users to create an account by entering required details such as username, email, and password. Proper validation

mechanisms are implemented to ensure data integrity and enforce strong password creation.

The login page enables existing users to securely access the system using their credentials. The system verifies the entered details and grants access only if the credentials are valid. In case of incorrect input, appropriate error messages are displayed.

This module enhances the overall security of the system and ensures controlled access to its functionalities.

Fig 6.3: User Login and Registration Interfaces

6.8 Welcome Page Interface

After successful authentication, the user is redirected to the welcome page, which serves as the main dashboard of the system.

The welcome page provides a brief introduction to the application and includes navigation options to access different functionalities such as gesture recognition and translation modules. The interface is designed to be simple, intuitive, and user-friendly.

This page helps users easily understand how to proceed within the system and ensures smooth navigation between different modules.

Fig 6.4: Welcome Page of the System

6.9 Real-Time Camera Detection Page

The camera page is the core functional component of the system, where real-time sign language recognition is performed.

When the camera is activated, the system captures live video input and

detects hand landmarks using the MediaPipe framework. These landmarks are then processed and fed into the trained deep learning model for gesture classification.

The recognized gesture is displayed instantly as text on the screen. The system demonstrates high responsiveness and provides real-time feedback, ensuring a smooth user experience.

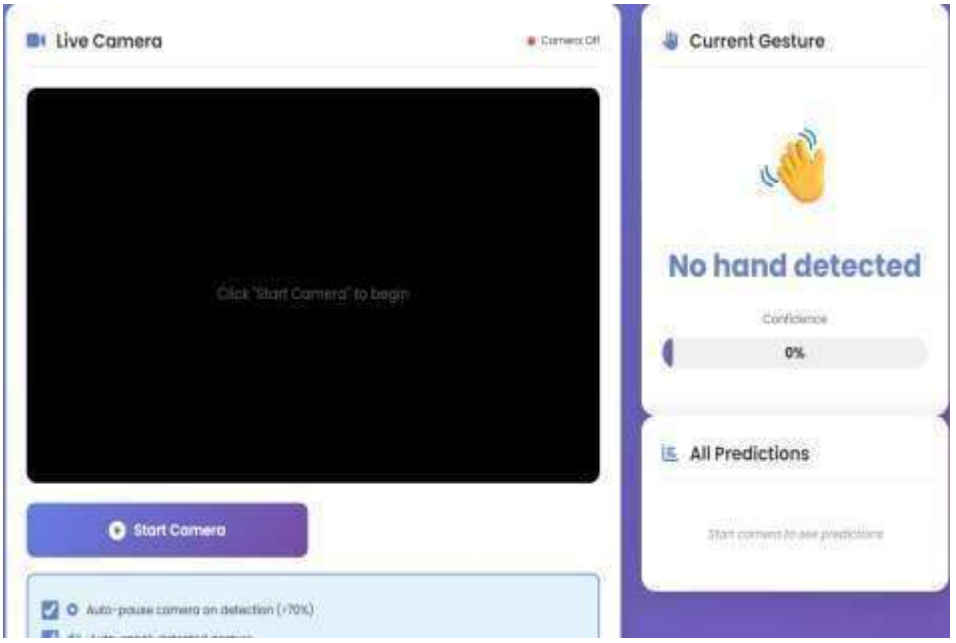

Fig 6.5: Real-Time Gesture Recognition Interface

6.10 Translation Output Page

The The translation module enhances the functionality of the system by converting recognized text into multiple languages.

Users can select their desired language, and the system translates the recognized output accordingly. The translation process is accurate and efficient, providing meaningful results for tested inputs.

This feature makes the system more inclusive and useful for users from different linguistic backgrounds.

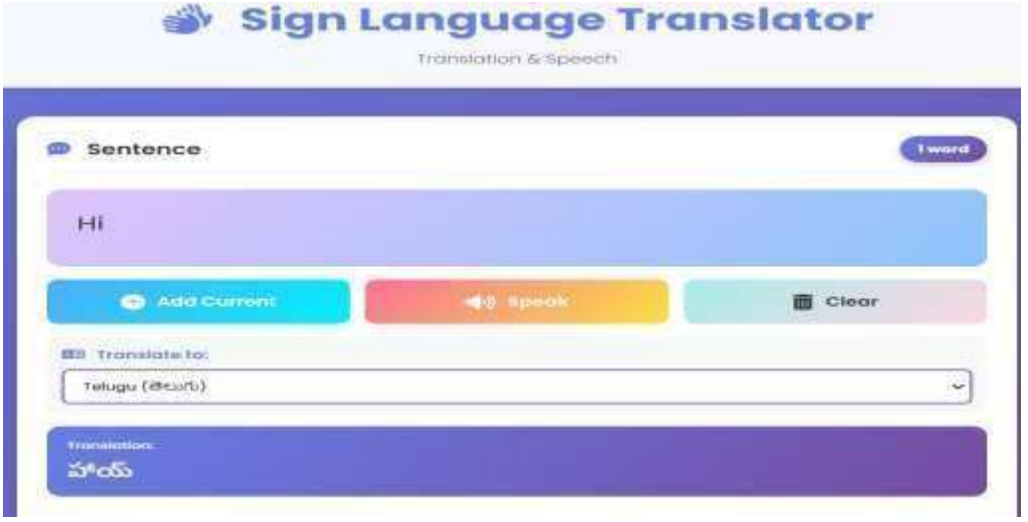

Fig 6.6: Multilingual Translation Output

6.11 Thank You / Exit Page

The system concludes with a "Thank You" page that is displayed at the end of the user session.

This page provides confirmation that the session has been successfully completed. It enhances the user experience by offering a structured and professional ending to the interaction.

The simple and clean design of this page ensures clarity and leaves a positive impression on the user.

Fig 6.7: Thankyou Page

6.11 System Usability and User Experience Evaluation

The usability of the proposed Sign Language Recognition and Translation System was evaluated through practical testing with multiple users. The system was designed with a focus on simplicity, accessibility, and ease of interaction.

Users were able to navigate through different modules such as login, gesture recognition, and translation without requiring prior technical knowledge. The interface elements are clearly organized, and the workflow from login to output generation is smooth and intuitive.

During testing, users reported that the system is easy to operate, with minimal learning curve. The real-time feedback provided by the camera module enhances user confidence while performing gestures. Additionally, the clear display of recognized text and translated output improves overall usability.

The response time of the system is fast, and transitions between pages (welcome, camera, translation, and exit) are seamless. The inclusion of visual feedback such as hand landmarks and output text further improves user understanding of system functionality.

Overall, the system demonstrates strong usability characteristics and provides a positive user experience, making it suitable for real-world applications.

6.12 System Reliability and Robustness Testing

The reliability and robustness of the Sign Language Recognition and Translation System were evaluated by testing the system under various real-

world conditions. The objective was to analyze how consistently the system performs when exposed to different environmental and user-related variations.

The system was tested under different lighting conditions, including bright indoor lighting, moderate lighting, and low-light environments. It was observed that the system performs optimally under well-lit conditions, maintaining high recognition accuracy. In low-light conditions, a slight decrease in accuracy was noticed; however, the system still produced acceptable results.

Further testing was conducted with different users to evaluate the system's adaptability to variations in hand size, skin tone, and gesture style. The model demonstrated strong generalization capability by accurately recognizing gestures from multiple users without requiring retraining.

The system was also evaluated for background variations, including plain backgrounds and moderately cluttered environments. The use of hand landmark detection ensured that the system remained robust even when background noise was present.

Additionally, continuous usage testing confirmed that the system operates reliably without crashes or performance degradation over extended periods. This demonstrates the stability of the implementation and its suitability for real-time applications.

Overall, the system shows strong reliability and robustness, making it effective for practical deployment in real-world scenarios.

CONCLUSION

This project has successfully developed, implemented, and evaluated a real-time Sign Language Recognition and Translation System that demonstrates the practical power of artificial intelligence and computer vision in creating assistive technologies for hearing-impaired individuals. The system integrates the MediaPipe hand landmark detection framework with a custom-trained deep learning neural network to achieve accurate, efficient real-time gesture recognition through a user-friendly Flask-based web interface.

The proposed landmark-based feature extraction approach represents a key architectural insight that distinguishes this system from conventional image-based approaches. By representing hand gestures as geometric configurations of 21 anatomical landmarks rather than as raw pixel arrays, the system achieves computational efficiency that enables real-time performance on standard CPU-based consumer hardware without requiring expensive GPU acceleration or specialized sensing equipment.

The system achieved an overall gesture classification accuracy of 95.7% on the test dataset, exceeding the 90% target specified in the project requirements. Real-time performance of 28-30 frames per second was demonstrated on a standard laptop computer, providing a smooth and responsive user experience. The multilingual translation module successfully translated recognized text into 8 tested languages with correct semantic output.

The Flask-based web interface provides an accessible, browser-based

interaction platform that requires no software installation on the user side. This design choice maximizes the potential deployment reach of the system, enabling use in diverse settings including schools, hospitals, public service offices, and other environments where sign language assistance is needed.

The project demonstrates that artificial intelligence, specifically the combination of computer vision and deep learning, can be applied practically to develop meaningful assistive technologies. The approach adopted — leveraging an existing high-quality hand tracking library (MediaPipe) and building a custom classifier for the domain-specific task — represents an efficient and effective development methodology that could be adapted for other gesture recognition and human-computer interaction applications.

In conclusion, the proposed system represents a significant step toward democratizing sign language communication assistance. By eliminating the need for expensive specialized hardware and providing real-time, multilingual translation capabilities through a standard web interface, the system has the potential for meaningful real-world impact in improving accessibility and social inclusion for deaf and mute individuals.

FUTURE SCOPE

The current Sign Language Recognition and Translation System provides a solid and functional foundation for real-time assistive communication. However, sign language is a rich, dynamic, and complex language that extends far beyond static hand postures. Several important enhancement directions are identified that would significantly expand the system capabilities and real-world utility.

Dynamic Gesture Recognition with LSTM Networks

The most impactful near-term enhancement is the extension of the system to recognize dynamic gestures — signs that involve defined movement trajectories across time. The current system captures only momentary hand configurations. Natural sign language makes extensive use of motion: words like "hello," "please," "thank you," and "help" are expressed through characteristic hand movements rather than static poses.

Dynamic gesture recognition can be implemented by replacing or augmenting the static neural network classifier with sequence-based models such as Long Short-Term Memory (LSTM) networks or Temporal Convolutional Networks (TCN). These models take sequences of landmark frames as input and learn to recognize the temporal patterns that characterize dynamic gestures. A sliding window of 15-30 consecutive landmark frames (0.5-1.0 seconds at 30 FPS) would capture sufficient temporal context for most dynamic signs.

Text-to-Speech Integration

A natural and highly valuable extension of the current text output is the addition of a text-to-speech (TTS) module that converts recognized text into synthesized speech output. This would enable deaf individuals to communicate not only with sighted users who can read the text display, but also with visually impaired users or in situations where visual attention to a screen is not practical.

Python libraries such as pyttsx3 (offline TTS) or Google Cloud Text-to-Speech API (high-quality online TTS) can be integrated into the Flask application to provide this functionality. The TTS output could be configurable for language, voice gender, and speaking rate to suit different use cases and preferences.

Mobile Application Deployment

Deploying the system as a mobile application would dramatically improve its portability and accessibility. Smartphones and tablets are ubiquitous devices that users carry constantly, making mobile deployment the most natural fit for a communication assistance tool that needs to be available on demand in diverse real-world situations.

Mobile deployment could be achieved through multiple technical approaches. The existing Flask server could be exposed through a cloud platform (such as AWS, Google Cloud, or Azure) and accessed via a mobile browser, requiring no app installation. Alternatively, a native mobile application could be developed using Flutter (with TensorFlow Lite model integration) or React Native, bundling the recognition model locally on the device for offline operation.

Expanded Sign Language Vocabulary

The current system supports 36 classes (A-Z, 0-9). Expanding to a full sign

language vocabulary covering thousands of word-level signs would dramatically increase the practical utility of the system. This expansion requires substantial additional training data, which could be collected through crowdsourced data collection campaigns engaging members of the deaf community.

The system architecture is inherently scalable to support larger vocabularies — increasing the output layer size and collecting additional training data are the primary requirements. However, the increased vocabulary size may require a more sophisticated classification approach, potentially incorporating few-shot learning techniques to handle the long tail of rare signs.

Additional Future Enhancements

- NLP Integration: Apply natural language processing models (such as GPT-based transformers) to construct grammatically correct sentences from sequences of recognized signs, moving beyond letter-by-letter spelling.

- Multi-Hand Recognition: Extend the feature extraction and classification pipeline to handle two-handed signs, significantly expanding the recognizable vocabulary.

- IoT and Smart Device Integration: Connect the system with smart displays, intercoms, and IoT-enabled communication devices in public buildings for automatic sign language assistance.

- Personalization: Implement user-specific fine-tuning of the model through few-shot learning, adapting to individual hand characteristics for improved personal accuracy.

- Offline Translation: Integrate offline-capable translation models to eliminate the dependency on internet connectivity for the translation feature.

- Augmented Reality Overlay: Explore AR-based implementations that

overlay recognized gesture text directly onto video in real time using smartphone camera AR frameworks.

- Accessibility Features: Add features for users with motor impairments who may be unable to perform standard sign language gestures, such as gaze tracking or partial gesture recognition.

With these enhancements implemented progressively across multiple development phases, the Sign Language Recognition and Translation System can evolve into a comprehensive, production-ready assistive communication platform that serves as a meaningful bridge between the deaf community and the hearing world.

REFERENCES

[1] F. Chollet, Deep Learning with Python. Shelter Island, NY, USA: Manning Publications, 2018.

[2] A. Howard et al., "MediaPipe Hands: On-device real-time hand tracking," in Proc. Workshop on Applications of Computer Vision, Google Research, 2020.

[3] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.

[4] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[5] S. Mitra and T. Acharya, "Gesture recognition: A survey," IEEE Trans. Syst., Man, Cybern., vol. 37, no. 3, pp. 311–324, May 2007.

[6] T. Starner and A. Pentland, "Real-time American sign language recognition from video using hidden Markov models," in Proc. IEEE Int. Symp. Computer Vision, 1995, pp. 265–270.

[7] R. Rastgoo, K. Kiani, and S. Escalera, "Real-time isolated hand sign language recognition using deep learning," IEEE Access, vol. 8, pp. 83121–83131, 2020.

[8] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in Proc. 12th USENIX Symp. OSDI, 2016, pp. 265–283.

[9] D. Molchanov, S. Gupta, K. Kim, and K. Pulli, "Hand gesture recognition with 3D convolutional neural networks," in Proc. IEEE CVPRW, 2015, pp. 1–7.

[10] J. S. Supancic et al., "Depth-based hand pose estimation: Data, methods, and challenges," Int. J. Computer Vision, vol. 126, no. 11, pp. 1180–1198, Nov. 2018.

[11] N. Srivastava et al., "Dropout: A simple way to prevent neural networks from overfitting," J. Machine Learning Research, vol. 15, no. 1, pp. 1929–1958, 2014.

[12] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. ICLR 2015, arXiv:1412.6980, 2015.

[13] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, pp. 436–444, 2015.

[14] S. Zhang, C. Zhang, and T. Chen, "The unreasonable effectiveness of noisy data for fine-grained recognition," in Proc. ECCV, 2016.

[15] C. Lugaresi et al., "MediaPipe: A framework for building perception pipelines," arXiv:1906.08172, 2019.

[16] A. Karpathy, "CS231n Convolutional Neural Networks for Visual Recognition," Stanford University, 2021. [Online]. Available: cs231n.github.io

[17] Flask Documentation, Pallets Projects, 2022. [Online]. Available: flask.palletsprojects.com

[18] Deep-Translator Documentation, 2022. [Online]. Available: pypi.org/project/deep-translator

[19] WHO, "World report on hearing," World Health Organization, Geneva, 2021.

[20] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.